

CLUSTERING

Complete Assignment Answers

PART A – THEORETICAL QUESTIONS

Question 1: What is Unsupervised Learning in the Context of Machine Learning?

Answer:

Unsupervised Learning is a type of Machine Learning in which the model learns patterns and structures from data without using predefined target labels.

In supervised learning, the dataset contains input features and a target variable. In unsupervised learning, only the input data is generally available.

Main Applications:

- Clustering
- Dimensionality Reduction
- Anomaly Detection
- Pattern Discovery

Example:

Suppose an e-commerce company has customer information such as:

- Age
- Income

- Spending amount
- Number of purchases

There may be no predefined customer category. A clustering algorithm can automatically group customers into similar groups.

Easy Definition:

Unsupervised Learning finds hidden patterns or groups in unlabeled data.

Question 2: How Does K-Means Clustering Algorithm Work?

Answer:

K-Means is an unsupervised clustering algorithm that divides data into a predefined number of clusters, represented by **K**.

Steps:

1. Choose the number of clusters K .
2. Initialize K cluster centroids.
3. Assign every data point to the nearest centroid.
4. Calculate the new centroid of each cluster.
5. Repeat assignment and centroid calculation.
6. Stop when the centroids or assignments no longer change significantly.

Example:

Data Points

↓

Choose K



Initialize Centroids



Assign Points to Nearest Centroid



Calculate New Centroids



Repeat



Final Clusters

Objective:

K-Means tries to minimize the within-cluster sum of squared distances, commonly called **inertia**.

Easy Definition:

K-Means groups data points into K clusters based on their distance from cluster centroids.

Question 3: Explain the Concept of a Dendrogram in Hierarchical Clustering.

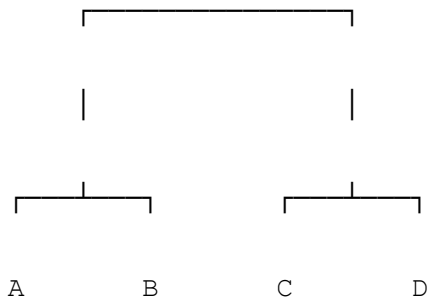
Answer:

A **dendrogram** is a tree-like diagram that represents how observations or clusters are merged or separated during hierarchical clustering.

In **agglomerative hierarchical clustering**, every data point starts as an individual cluster. The closest clusters are then repeatedly merged.

The height at which two clusters are joined represents their dissimilarity according to the selected linkage and distance measure.

Example:



By cutting the dendrogram at a selected height, we can obtain a desired number of clusters.

Easy Definition:

A dendrogram is a tree diagram showing the hierarchy of cluster merging.

Question 4: What is the Main Difference Between K-Means and Hierarchical Clustering?

Answer:

K-Means	Hierarchical Clustering
Requires K to be specified	Does not require K initially
Uses cluster centroids	Uses distances and linkage
Iteratively updates centroids	Builds a hierarchy
Usually computationally efficient	Can be more computationally expensive
Produces flat clusters	Produces a hierarchy of clusters
Sensitive to initialization	No centroid initialization required
Works well with roughly compact clusters	Can represent hierarchical relationships

Easy Definition:

K-Means creates K flat clusters, while Hierarchical Clustering builds a hierarchy of clusters.

Question 5: What are the Advantages of DBSCAN over K-Means?

Answer:

DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise.

Advantages of DBSCAN over K-Means include:

1. It does not require the number of clusters to be specified in advance.
2. It can identify noise/outliers.
3. It can find arbitrarily shaped clusters.
4. It is based on data density rather than centroids.
5. It can work well when clusters are irregularly shaped.

Example:

K-Means may struggle with:

C-shaped cluster

)

)

)

DBSCAN can often identify such density-connected structures more naturally.

Limitation:

DBSCAN can struggle when clusters have very different densities or when the data is high-dimensional.

Easy Definition:

DBSCAN can find irregularly shaped clusters and identify noise without specifying the number of clusters.

Question 6: When Would You Use Silhouette Score in Clustering?

Answer:

The **Silhouette Score** is used to evaluate the quality of clustering.

It measures:

- How similar a data point is to its own cluster.
- How separated it is from neighboring clusters.

The score ranges from **-1 to +1**.

Interpretation:

Score	Meaning
Near +1	Good clustering
Around 0	Overlapping/boundary points
Negative	Point may be assigned to the wrong cluster

Formula:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Where:

- $a(i)$ = average distance of point i from points in its own cluster.
- $b(i)$ = lowest average distance from points in another cluster.

Easy Definition:

Silhouette Score measures how well-separated and internally cohesive the clusters are.

Question 7: What are the Limitations of Hierarchical Clustering?

Answer:

Major limitations include:

1. Can be computationally expensive for large datasets.
2. Requires storing and processing many pairwise distances.
3. Once a merge or split is made, standard hierarchical algorithms generally do not undo it.
4. Sensitive to the choice of distance metric.
5. Sensitive to the choice of linkage method.
6. Can be affected by noise and outliers.

Easy Definition:

Hierarchical clustering can be expensive and its result depends strongly on distance and linkage choices.

Question 8: Why is Feature Scaling Important in Clustering Algorithms Like K-Means?

Answer:

Feature scaling is important because K-Means commonly uses distance calculations such as Euclidean distance.

Suppose we have:

Age: 20–60

Income: 20,000–500,000

Income has a much larger numerical scale than age.

Without scaling, income may dominate the distance calculation.

Common Scaling Methods:

- StandardScaler
- MinMaxScaler

Example:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Easy Definition:

Feature scaling prevents large-scale features from dominating distance-based clustering.

Question 9: How Does DBSCAN Identify Noise Points?

Answer:

DBSCAN classifies points based on the density of neighboring observations.

If a point does not have enough neighboring points within the specified radius, it may be classified as **noise**.

The two important parameters are:

- `eps`
- `min_samples`

DBSCAN uses the label `-1` to represent noise points in Scikit-learn.

Example:

```
from sklearn.cluster import DBSCAN
```

```
model = DBSCAN(  
  
    eps=0.3,  
  
    min_samples=5  
)
```

```
labels = model.fit_predict(X)
```

Noise points:

```
noise = X[labels == -1]
```

Easy Definition:

DBSCAN identifies low-density points that do not belong to a sufficiently dense cluster as noise.

Question 10: Define Inertia in the Context of K-Means.

Answer:

Inertia measures the sum of squared distances between each data point and the centroid of its assigned cluster.

Formula:

Inertia=

$$\sum_{i=1}^n ||$$

$$\sum_{i=1}^n \frac{1}{2} \|x_i - \mu_c\|^2$$

Where:

- (x_i) = data point
- (μ_{c_i}) = centroid of its assigned cluster

Interpretation:

Lower inertia generally means points are closer to their assigned centroids.

However, inertia will generally decrease as K increases, so it should not be used alone to determine the best number of clusters.

Python:

```
from sklearn.cluster import KMeans
```

```
model = KMeans(

    n_clusters=3,

    random_state=42,

    n_init=10
```

```
)
```

```
model.fit(X)
```

```
print(model.inertia_)
```

Easy Definition:

Inertia measures how close data points are to their assigned K-Means centroids.

Question 11: What is the Elbow Method in K-Means Clustering?

Answer:

The **Elbow Method** is a technique used to help choose a suitable value of K.

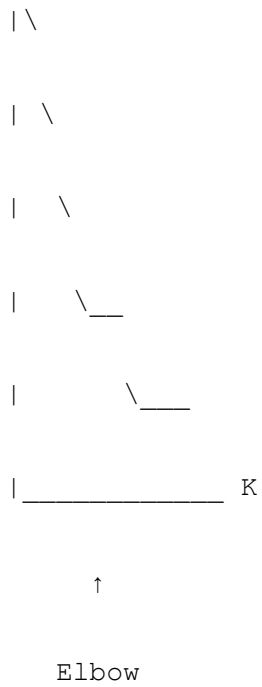
We calculate inertia for different values of K and plot the results.

As K increases, inertia decreases. We look for the point where the improvement begins to slow down significantly. This point resembles an **elbow**.

Example:

Inertia

|



Python:

```
inertias = []

for k in range(2, 10):

    model = KMeans(

        n_clusters=k,

        random_state=42,

        n_init=10

    )

    model.fit(X)
```

```
inertias.append(model.inertia_)
```

Easy Definition:

The Elbow Method selects K by looking for a point where additional clusters give diminishing improvement in inertia.

Question 12: Describe the Concept of "Density" in DBSCAN.

Answer:

In DBSCAN, **density** refers to how many data points are located within a specified neighborhood around a point.

The neighborhood is controlled by `eps`, and the minimum number of points needed to form a dense region is controlled by `min_samples`.

High Density:

Many points are close together.

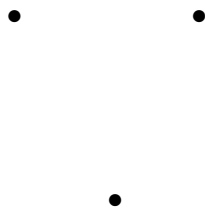
• • •

• •

• • •

Low Density:

Few points are close together.



DBSCAN identifies connected high-density regions as clusters.

Easy Definition:

Density in DBSCAN means the concentration of data points within a specified neighborhood.

Question 13: Can Hierarchical Clustering Be Used on Categorical Data?

Answer:

Yes, hierarchical clustering can be used with categorical data, but the standard Euclidean distance is not appropriate for purely categorical variables.

For categorical data, an appropriate dissimilarity measure such as **Hamming distance** or other categorical-data measures should be used.

The categorical variables may also need suitable preprocessing depending on the chosen distance method.

Example:

For data:

```
Gender | City | Product
```

```
Male   | Delhi | A
```

```
Female | Mumbai | B
```

We can use an appropriate categorical distance measure and then perform hierarchical clustering.

Important:

Simply applying Euclidean distance to raw category labels such as:

```
Delhi = 1
```

```
Mumbai = 2
```

```
Jaipur = 3
```

can introduce artificial numerical relationships.

Easy Definition:

Hierarchical clustering can handle categorical data when an appropriate categorical distance measure is used.

Question 14: What Does a Negative Silhouette Score Indicate?

Answer:

A **negative Silhouette Score** generally indicates that a data point is closer, on average, to another cluster than to the cluster it has been assigned to.

This may indicate:

- Poor cluster assignment.
- Overlapping clusters.
- Incorrect number of clusters.
- Weak cluster separation.

Example:

`Silhouette = -0.2`

This suggests that the point may fit better in another cluster.

Easy Definition:

A negative Silhouette Score indicates that a point may be assigned to the wrong or overlapping cluster.

Question 15: Explain the Term "Linkage Criteria" in Hierarchical Clustering.

Answer:

Linkage criteria determine how the distance between two clusters is calculated.

Common linkage methods include:

1. Single Linkage

Uses the minimum distance between points in two clusters.

2. Complete Linkage

Uses the maximum distance between points in two clusters.

3. Average Linkage

Uses the average pairwise distance between points in the two clusters.

4. Ward Linkage

Merges clusters to minimize the increase in within-cluster variance.

Easy Definition:

Linkage determines how the distance between clusters is measured during hierarchical clustering.

Question 16: Why Might K-Means Clustering Perform Poorly on Data with Varying Cluster Sizes or Densities?

Answer:

K-Means assumes that clusters are reasonably compact and roughly spherical around their centroids.

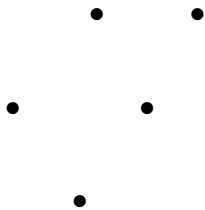
If clusters have:

- Different sizes
- Different densities
- Irregular shapes

K-Means may produce poor assignments.

Example:

Large sparse cluster



Small dense cluster



K-Means may divide the data according to distance to centroids rather than the actual density structure.

Alternative:

DBSCAN or other clustering methods may be more suitable depending on the data.

Easy Definition:

K-Means works best for compact, roughly spherical clusters and can struggle with irregular sizes or densities.

Question 17: What are the Core Parameters in DBSCAN, and How Do They Influence Clustering?

Answer:

The two main parameters are:

1. `eps`

Defines the radius of the neighborhood around each point.

- Small `eps` → smaller neighborhoods and potentially more noise.
- Large `eps` → larger neighborhoods and potentially merged clusters.

2. `min_samples`

Defines the minimum number of samples required in an `eps` neighborhood for a point to qualify as a core point.

- Higher value → requires denser regions.
- Lower value → allows less dense regions to form clusters.

Python:

```
model = DBSCAN(  
  
    eps=0.5,  
  
    min_samples=5  
)
```

Easy Definition:

`eps` controls neighborhood size, while `min_samples` controls the minimum density required for core points.

Question 18: How Does K-Means++ Improve Upon Standard K-Means Initialization?

Answer:

K-Means++ provides a smarter initialization strategy for selecting initial cluster centroids.

Instead of selecting all initial centroids randomly, K-Means++ chooses initial centers so that new centers tend to be farther from already selected centers.

Benefits:

- Better starting positions.
- Often faster convergence.
- Can produce better clustering solutions.
- Reduces the chance of poor initialization.

Scikit-learn's KMeans uses K-Means++ by default.

```
model = KMeans(  
  
    n_clusters=3,  
  
    init="k-means++",  
  
    random_state=42,  
  
    n_init=10  
)
```

Easy Definition:

K-Means++ chooses better-separated initial centroids to improve K-Means initialization.

Question 19: What is Agglomerative Clustering?

Answer:

Agglomerative Clustering is a bottom-up hierarchical clustering algorithm.

Initially, every data point is considered its own cluster.

Then the closest clusters are repeatedly merged until the desired hierarchy is obtained.

Process:

Individual Points

↓

Small Clusters

↓

Larger Clusters

↓

Final Hierarchy

Python:

```
from sklearn.cluster import AgglomerativeClustering
```

```
model = AgglomerativeClustering(
```

```
    n_clusters=3,
```

```
    linkage="ward"
```

```
)
```

```
labels = model.fit_predict(X)
```

Easy Definition:

Agglomerative clustering starts with individual points and repeatedly merges the closest clusters.

Question 20: What Makes Silhouette Score a Better Metric Than Just Inertia for Model Evaluation?

Answer:

Inertia only measures how close points are to their own K-Means centroids.

A lower inertia is usually obtained by increasing the number of clusters, so inertia alone does not directly measure whether clusters are well separated.

Silhouette Score considers both:

- **Cohesion** within the cluster.
- **Separation** from other clusters.

Therefore, Silhouette Score can provide more useful information about cluster quality.

Comparison:

Inertia

Silhouette Score

Measures within-cluster squared distance

Measures cohesion and separation

Lower is better

Higher is generally better

Always tends to decrease as K increases

Can increase or decrease

Mainly useful for K-Means

Can be used with many clustering algorithms

Easy Definition:

Silhouette Score evaluates both how compact and how well-separated the clusters are.

PART B – PRACTICAL QUESTIONS

Question 21: Generate Synthetic Data with 4 Centers Using `make_blobs` and Apply K-Means Clustering. Visualize Using a Scatter Plot.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

# Generate data
```

```
X, y = make_blobs(  
  
    n_samples=500,  
  
    centers=4,  
  
    cluster_std=1.0,  
  
    random_state=42  
  
)  
  
# K-Means  
  
model = KMeans(  
  
    n_clusters=4,  
  
    random_state=42,  
  
    n_init=10  
  
)  
  
labels = model.fit_predict(X)  
  
# Plot  
  
plt.figure(figsize=(8, 6))
```

```
plt.scatter(  
  
    X[:, 0],  
  
    X[:, 1],  
  
    c=labels,  
  
    cmap="viridis",  
  
    s=30  
  
)
```

```
plt.scatter(  
  
    model.cluster_centers_[:, 0],  
  
    model.cluster_centers_[:, 1],  
  
    marker="x",  
  
    s=200,  
  
    label="Centroids"  
  
)
```

```
plt.title("K-Means Clustering with 4 Centers")
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.legend()
```

```
plt.show()
```

Observation:

The data is divided into four clusters and the cluster centers are shown on the scatter plot.

Question 22: Load the Iris Dataset and Use Agglomerative Clustering to Group the Data into 3 Clusters. Display the First 10 Predicted Labels.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.cluster import AgglomerativeClustering


# Load Iris

iris = load_iris()


X = iris.data
```

```
# Agglomerative Clustering

model = AgglomerativeClustering(

    n_clusters=3,

    linkage="ward"

)

labels = model.fit_predict(X)

print("First 10 Predicted Labels:")

print(labels[:10])
```

Observation:

The Iris observations are divided into three clusters.

Cluster numbers such as 0, 1, and 2 are arbitrary labels and do not necessarily correspond directly to the original Iris class names.

Question 23: Generate Synthetic Data Using make_moons and Apply DBSCAN. Highlight Outliers in the Plot.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons

from sklearn.cluster import DBSCAN

# Generate moon data

X, y = make_moons(

    n_samples=500,

    noise=0.08,

    random_state=42

)

# DBSCAN

model = DBSCAN(

    eps=0.2,
```

```
        min_samples=5

    )

    labels = model.fit_predict(X)

    # Plot

    plt.figure(figsize=(8, 6))

    # Non-noise points

    mask = labels != -1

    plt.scatter(

        X[mask, 0],

        X[mask, 1],

        c=labels[mask],

        cmap="viridis",

        s=30

    )
```

```
# Noise points

plt.scatter(

    X[~mask, 0],

    X[~mask, 1],

    marker="x",

    s=70,

    label="Outliers"

)

plt.title("DBSCAN on make_moons")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.legend()

plt.show()
```

Observation:

DBSCAN can identify the curved moon-shaped structure and mark low-density observations as noise.

Question 24: Load the Wine Dataset and Apply K-Means Clustering After Standardizing the Features. Print the Size of Each Cluster.

Python Code:

```
from sklearn.datasets import load_wine

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

import numpy as np


# Load Wine dataset

wine = load_wine()


X = wine.data


# Standardize

scaler = StandardScaler()


X_scaled = scaler.fit_transform(X)
```

```
# K-Means

model = KMeans(

    n_clusters=3,

    random_state=42,

    n_init=10

)

labels = model.fit_predict(X_scaled)


# Cluster sizes

unique, counts = np.unique(

    labels,

    return_counts=True

)

print("Cluster Sizes:")

for cluster, count in zip(
```

```
        unique,  
        counts  
    ):  
  
        print(  
            f"Cluster {cluster}: {count}"  
        )
```

Observation:

The output shows how many Wine samples belong to each cluster.

Question 25: Use make_circles to Generate Synthetic Data and Cluster It Using DBSCAN. Plot the Result.

Python Code:

```
import matplotlib.pyplot as plt  
  
from sklearn.datasets import make_circles  
  
from sklearn.cluster import DBSCAN  
  
# Generate circles
```

```
X, y = make_circles(

    n_samples=500,

    factor=0.5,

    noise=0.05,

    random_state=42

)


# DBSCAN

model = DBSCAN(

    eps=0.15,

    min_samples=5

)


labels = model.fit_predict(X)


# Plot

plt.figure(figsize=(8, 6))


plt.scatter(
```

```
X[:, 0],  
  
X[:, 1],  
  
c=labels,  
  
cmap="viridis",  
  
s=30  
  
)  
  
plt.title("DBSCAN on Concentric Circles")  
  
plt.xlabel("Feature 1")  
  
plt.ylabel("Feature 2")  
  
plt.show()
```

Observation:

DBSCAN is suitable for this type of non-linearly shaped data because it is density-based.

Question 26: Load the Breast Cancer Dataset, Apply MinMaxScaler, and Use K-Means with 2 Clusters. Output the Cluster Centroids.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.preprocessing import MinMaxScaler

from sklearn.cluster import KMeans


# Load dataset

data = load_breast_cancer()


X = data.data


# Scaling

scaler = MinMaxScaler()


X_scaled = scaler.fit_transform(X)


# K-Means

model = KMeans(

    n_clusters=2,
```

```
        random_state=42,  
  
        n_init=10  
    )  
  
model.fit(X_scaled)  
  
print("Cluster Centroids:")  
  
print(model.cluster_centers_)
```

Observation:

The centroids represent the average standardized feature values of the two clusters.

Question 27: Generate Synthetic Data Using make_blobs with Varying Cluster Standard Deviations and Cluster with DBSCAN.

Python Code:

```
import numpy as np  
  
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_blobs
```

```
from sklearn.cluster import DBSCAN
```

```
# Generate blobs
```

```
X1, _ = make_blobs(  
  
    n_samples=200,  
  
    centers=[[-4, 0]],  
  
    cluster_std=0.5,  
  
    random_state=42  
  
)
```

```
X2, _ = make_blobs(  
  
    n_samples=200,  
  
    centers=[[0, 0]],  
  
    cluster_std=1.5,  
  
    random_state=43  
  
)
```

```
x3, _ = make_blobs(  
  
    n_samples=200,  
  
    centers=[[4, 0]],  
  
    cluster_std=0.7,  
  
    random_state=44  
  
)
```

```
X = np.vstack([  
  
    x1,  
  
    x2,  
  
    x3  
  
])
```

```
# DBSCAN
```

```
model = DBSCAN(  
  
    eps=0.6,  
  
    min_samples=5  
  
)
```

```
labels = model.fit_predict(X)

# Plot

plt.figure(figsize=(8, 6))

plt.scatter(

    X[:, 0],

    X[:, 1],

    c=labels,

    cmap="viridis",

    s=30

)

plt.title("DBSCAN with Varying Cluster Densities")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()
```

Observation:

DBSCAN can have difficulty when different clusters have substantially different densities because one global `eps` value may not suit every cluster.

Question 28: Load the Digits Dataset, Reduce It to 2D Using PCA, and Visualize Clusters from K-Means.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.cluster import KMeans

# Load Digits

digits = load_digits()

X = digits.data
```

```
# Scale

X_scaled = StandardScaler().fit_transform(X)


# PCA

pca = PCA(

    n_components=2,

    random_state=42

)

X_pca = pca.fit_transform(X_scaled)


# K-Means

model = KMeans(

    n_clusters=10,

    random_state=42,

    n_init=10

)

labels = model.fit_predict(X_pca)
```

```
# Plot

plt.figure(figsize=(9, 7))

plt.scatter(

    X_pca[:, 0],

    X_pca[:, 1],

    c=labels,

    cmap="tab10",

    s=20

)

plt.title("K-Means Clustering on Digits after PCA")

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.show()
```

Observation:

PCA reduces the high-dimensional digit features to two dimensions for visualization.

Question 29: Create Synthetic Data Using `make_blobs` and Evaluate Silhouette Scores for $K = 2$ to 5. Display as a Bar Chart.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score

# Generate data

X, y = make_blobs(

    n_samples=500,

    centers=4,

    cluster_std=1.0,

    random_state=42

)
```

```
k_values = range(2, 6)
```

```
scores = []
```

```
for k in k_values:
```

```
    model = KMeans(
```

```
        n_clusters=k,
```

```
        random_state=42,
```

```
        n_init=10
```

```
    )
```

```
    labels = model.fit_predict(X)
```

```
    score = silhouette_score(
```

```
        X,
```

```
        labels
```

```
    )
```

```
scores.append(score)

# Bar chart

plt.figure(figsize=(8, 5))

plt.bar(

    list(k_values),

    scores

)

plt.xlabel("Number of Clusters (K)")

plt.ylabel("Silhouette Score")

plt.title("Silhouette Scores for K=2 to K=5")

plt.show()

for k, score in zip(

    k_values,

    scores
```

```
) :  
  
    print(  
  
        f"K={k}: Silhouette Score={score:.4f}"  
  
    )
```

Observation:

The K value with the highest Silhouette Score is generally preferred among the tested values.

Question 30: Load the Iris Dataset and Use Hierarchical Clustering to Group Data. Plot a Dendrogram with Average Linkage.

Python Code:

```
import matplotlib.pyplot as plt  
  
from sklearn.datasets import load_iris  
  
from scipy.cluster.hierarchy import (  
  
    dendrogram,
```

```
        linkage

    )

# Load Iris

iris = load_iris()

X = iris.data

# Average linkage

linked = linkage(

    X,

    method="average"

)

# Dendrogram

plt.figure(figsize=(12, 7))

dendrogram(

    linked
```

```
)

plt.title(

    "Iris Hierarchical Clustering Dendrogram"

)

plt.xlabel("Sample Index")

plt.ylabel("Distance")

plt.show()
```

Observation:

The dendrogram shows how the Iris samples are progressively merged into larger clusters.

Question 31: Generate Synthetic Data with Overlapping Clusters Using `make_blobs`, Then Apply K-Means and Visualize with Decision Boundaries.

Python Code:

```
import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans


# Overlapping data

X, y = make_blobs(

    n_samples=500,

    centers=3,

    cluster_std=2.5,

    random_state=42

)


# K-Means

model = KMeans(

    n_clusters=3,

    random_state=42,

    n_init=10
```

```
)
```

```
labels = model.fit_predict(X)
```

```
# Plot points
```

```
plt.figure(figsize=(9, 7))
```

```
plt.scatter(
```

```
    X[:, 0],
```

```
    X[:, 1],
```

```
    c=labels,
```

```
    cmap="viridis",
```

```
    s=25
```

```
)
```

```
# Centroids
```

```
centers = model.cluster_centers_
```

```
plt.scatter(
```

```

        centers[:, 0],

        centers[:, 1],

        marker="X",

        s=250,

        label="Centroids"

    )

# Approximate decision regions

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1

y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1


xx, yy = np.meshgrid(

    np.linspace(x_min, x_max, 300),

    np.linspace(y_min, y_max, 300)

)


grid = np.c_[

    xx.ravel(),

    yy.ravel()

```

```
]

grid_labels = model.predict(grid)

plt.contourf(

    xx,

    yy,

    grid_labels.reshape(xx.shape),

    alpha=0.15,

    cmap="viridis"

)

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.title("K-Means with Decision Regions")

plt.legend()

plt.show()
```

Observation:

The decision regions illustrate the areas where K-Means assigns points to different nearest centroids.

Question 32: Load the Digits Dataset and Apply DBSCAN After Reducing Dimensions with t-SNE. Visualize the Results.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits

from sklearn.preprocessing import StandardScaler

from sklearn.manifold import TSNE

from sklearn.cluster import DBSCAN

# Load Digits

digits = load_digits()

X = digits.data
```

```
# Scale

X_scaled = StandardScaler().fit_transform(X)


# t-SNE

tsne = TSNE(

    n_components=2,

    random_state=42,

    perplexity=30

)


X_tsne = tsne.fit_transform(X_scaled)


# DBSCAN

model = DBSCAN(

    eps=3.0,

    min_samples=5

)
```

```
labels = model.fit_predict(X_tsne)

# Plot

plt.figure(figsize=(9, 7))

plt.scatter(

    X_tsne[:, 0],

    X_tsne[:, 1],

    c=labels,

    cmap="tab10",

    s=15

)

plt.title("DBSCAN on Digits after t-SNE")

plt.xlabel("t-SNE 1")

plt.ylabel("t-SNE 2")

plt.show()
```

Observation:

t-SNE is mainly used here for visualization/dimensionality reduction. DBSCAN is then applied in the resulting two-dimensional representation.

Question 33: Generate Synthetic Data Using `make_blobs` and Apply Agglomerative Clustering with Complete Linkage. Plot the Result.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs

from sklearn.cluster import AgglomerativeClustering

# Generate data

X, y = make_blobs(

    n_samples=500,

    centers=4,

    cluster_std=1.0,

    random_state=42
```

```
)
```

```
# Agglomerative clustering
```

```
model = AgglomerativeClustering(
```

```
    n_clusters=4,
```

```
    linkage="complete"
```

```
)
```

```
labels = model.fit_predict(X)
```

```
# Plot
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(
```

```
    X[:, 0],
```

```
    X[:, 1],
```

```
    c=labels,
```

```
    cmap="viridis",
```

```
    s=30
```

```
)

plt.title(

    "Agglomerative Clustering - Complete Linkage"

)

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()
```

Question 34: Load the Breast Cancer Dataset and Compare Inertia Values for $K = 2$ to 6 Using K-Means. Show Results in a Line Plot.

Python Code:

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import KMeans
```

```
# Load dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
# Scale
```

```
X_scaled = StandardScaler().fit_transform(X)
```

```
k_values = range(2, 7)
```

```
inertias = []
```

```
for k in k_values:
```

```
    model = KMeans(
```

```
        n_clusters=k,
```

```
        random_state=42,

        n_init=10

    )

    model.fit(X_scaled)

    inertias.append(

        model.inertia_

    )

# Print values

for k, inertia in zip(

    k_values,

    inertias

):

    print(

        f"K={k}: Inertia={inertia:.2f}"

    )
```

```
# Plot

plt.figure(figsize=(8, 5))

plt.plot(

    list(k_values),

    inertias,

    marker="o"

)

plt.xlabel("Number of Clusters (K)")

plt.ylabel("Inertia")

plt.title("K-Means Inertia for K=2 to K=6")

plt.show()
```

Observation:

Inertia decreases as K increases. The plot can be inspected for an elbow point.

Question 35: Generate Synthetic Concentric Circles Using make_circles and Cluster Using Agglomerative Clustering with Single Linkage.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_circles

from sklearn.cluster import AgglomerativeClustering

# Generate circles

X, y = make_circles(

    n_samples=500,

    factor=0.5,

    noise=0.05,

    random_state=42

)

# Single linkage
```

```
model = AgglomerativeClustering(  
  
    n_clusters=2,  
  
    linkage="single"  
  
)
```

```
labels = model.fit_predict(X)
```

```
# Plot
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(  
  
    X[:, 0],  
  
    X[:, 1],  
  
    c=labels,  
  
    cmap="viridis",  
  
    s=30  
  
)
```

```
plt.title(
```

```
    "Agglomerative Clustering - Single Linkage"  
  
)  
  
plt.xlabel("Feature 1")  
  
plt.ylabel("Feature 2")  
  
  
plt.show()
```

Observation:

Single linkage can connect points through chains and can therefore represent certain non-convex structures better than centroid-based clustering.

Question 36: Use the Wine Dataset, Apply DBSCAN After Scaling the Data, and Count the Number of Clusters Excluding Noise.

Python Code:

```
import numpy as np
```

```
from sklearn.datasets import load_wine

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import DBSCAN


# Load Wine dataset

wine = load_wine()


X = wine.data


# Scale

X_scaled = StandardScaler().fit_transform(X)


# DBSCAN

model = DBSCAN(

    eps=1.5,

    min_samples=5

)


labels = model.fit_predict(
```

```
        X_scaled

    )

    # Count clusters excluding noise

    clusters = set(labels)

    if -1 in clusters:

        clusters.remove(-1)

    print(

        "Number of clusters:",

        len(clusters)

    )

    print(

        "Number of noise points:",

        np.sum(labels == -1)

    )
```

Observation:

The exact number of clusters depends strongly on the chosen `eps` and `min_samples` values.

Question 37: Generate Synthetic Data with `make_blobs` and Apply KMeans. Then Plot the Cluster Centers on Top of the Data Points.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

# Generate data

X, y = make_blobs(

    n_samples=500,

    centers=4,

    cluster_std=1.0,
```

```
        random_state=42

    )

# K-Means

model = KMeans(

    n_clusters=4,

    random_state=42,

    n_init=10

)

labels = model.fit_predict(X)

# Plot points

plt.figure(figsize=(8, 6))

plt.scatter(

    X[:, 0],

    X[:, 1],

    c=labels,
```

```
cmap="viridis",

s=30

)

# Plot centers

centers = model.cluster_centers_

plt.scatter(

    centers[:, 0],

    centers[:, 1],

    marker="x",

    s=250,

    label="Cluster Centers"

)

plt.title("K-Means Cluster Centers")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.legend()
```

```
plt.show()
```

Question 38: Load the Iris Dataset, Cluster with DBSCAN, and Print How Many Samples Were Identified as Noise.

Python Code:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import DBSCAN

# Load Iris

iris = load_iris()

X = iris.data
```

```
# Scale

X_scaled = StandardScaler().fit_transform(X)


# DBSCAN

model = DBSCAN(

    eps=0.7,

    min_samples=5

)


labels = model.fit_predict(

    X_scaled

)


# Noise count

noise_count = np.sum(

    labels == -1

)
```

```
print(  
    "Number of noise samples:",  
    noise_count  
)
```

Observation:

The noise count depends on the chosen DBSCAN parameters.

Question 39: Generate Synthetic Non-Linearly Separable Data Using `make_moons`, Apply K-Means, and Visualize the Clustering Result.

Python Code:

```
import matplotlib.pyplot as plt  
  
from sklearn.datasets import make_moons  
  
from sklearn.cluster import KMeans  
  
# Generate moon data
```

```
X, y = make_moons(  
  
    n_samples=500,  
  
    noise=0.08,  
  
    random_state=42  
  
)  
  
# K-Means  
  
model = KMeans(  
  
    n_clusters=2,  
  
    random_state=42,  
  
    n_init=10  
  
)  
  
labels = model.fit_predict(X)  
  
# Plot  
  
plt.figure(figsize=(8, 6))  
  
plt.scatter(
```

```
X[:, 0],  
  
X[:, 1],  
  
c=labels,  
  
cmap="viridis",  
  
s=30  
  
)  
  
plt.title(  
  
    "K-Means on Non-Linearly Separable Data"  
  
)  
  
plt.xlabel("Feature 1")  
  
plt.ylabel("Feature 2")  
  
plt.show()
```

Observation:

K-Means may not correctly capture the curved moon-shaped structure because it is based on distance to centroids and generally favors compact, convex clusters.

Question 40: Load the Digits Dataset, Apply PCA to Reduce to 3 Components, Then Use KMeans and Visualize with a 3D Scatter Plot.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.cluster import KMeans

# Load Digits

digits = load_digits()

X = digits.data

# Scale
```

```
X_scaled = StandardScaler().fit_transform(X)
```

```
# PCA to 3 components
```

```
pca = PCA(
```

```
    n_components=3,
```

```
    random_state=42
```

```
)
```

```
X_pca = pca.fit_transform(
```

```
    X_scaled
```

```
)
```

```
# K-Means
```

```
model = KMeans(
```

```
    n_clusters=10,
```

```
    random_state=42,
```

```
    n_init=10
```

```
)
```

```
labels = model.fit_predict(
```

```
    X_pca
```

```
)
```

```
# 3D Plot
```

```
fig = plt.figure(
```

```
    figsize=(10, 8)
```

```
)
```

```
ax = fig.add_subplot(
```

```
    111,
```

```
    projection="3d"
```

```
)
```

```
ax.scatter(
```

```
    X_pca[:, 0],
```

```
    X_pca[:, 1],
```

```
    X_pca[:, 2],
```

```
    c=labels,
```

```
cmap="tab10",

s=15

)

ax.set_xlabel("PC1")

ax.set_ylabel("PC2")

ax.set_zlabel("PC3")

ax.set_title(

    "K-Means Clustering on Digits after 3D PCA"

)

plt.show()
```

Question 41: Generate Synthetic Blobs with 5 Centers and Apply KMeans. Then Use silhouette_score to Evaluate the Clustering.

Python Code:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score


# Generate data

X, y = make_blobs(

    n_samples=500,

    centers=5,

    cluster_std=1.0,

    random_state=42

)


# K-Means

model = KMeans(

    n_clusters=5,

    random_state=42,

    n_init=10
```

```
)

labels = model.fit_predict(X)

# Silhouette Score

score = silhouette_score(

    X,

    labels

)

print(

    "Silhouette Score:",

    score

)
```

Observation:

A higher Silhouette Score indicates better cohesion and separation for the resulting clusters.

Question 42: Load the Breast Cancer Dataset, Reduce Dimensionality Using PCA, and Apply Agglomerative Clustering. Visualize in 2D.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.cluster import AgglomerativeClustering

# Load dataset

data = load_breast_cancer()

X = data.data

# Scale

X_scaled = StandardScaler().fit_transform(X)
```

```
# PCA

pca = PCA(

    n_components=2,

    random_state=42

)

X_pca = pca.fit_transform(

    X_scaled

)


# Agglomerative Clustering

model = AgglomerativeClustering(

    n_clusters=2,

    linkage="ward"

)

labels = model.fit_predict(

    X_pca
```

```
)
```

```
# Plot
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(
```

```
    X_pca[:, 0],
```

```
    X_pca[:, 1],
```

```
    c=labels,
```

```
    cmap="viridis",
```

```
    s=30
```

```
)
```

```
plt.title(
```

```
    "Agglomerative Clustering after PCA"
```

```
)
```

```
plt.xlabel("Principal Component 1")
```

```
plt.ylabel("Principal Component 2")
```

```
plt.show()
```

Observation:

PCA reduces the original high-dimensional dataset to two dimensions, making cluster visualization possible.

Question 43: Generate Noisy Circular Data Using `make_circles` and Visualize Clustering Results from KMeans and DBSCAN Side-by-Side.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_circles

from sklearn.cluster import KMeans

from sklearn.cluster import DBSCAN

# Generate noisy circles
```

```
X, y = make_circles(  
  
    n_samples=500,  
  
    factor=0.5,  
  
    noise=0.08,  
  
    random_state=42  
  
)
```

```
# K-Means
```

```
kmeans = KMeans(  
  
    n_clusters=2,  
  
    random_state=42,  
  
    n_init=10  
  
)
```

```
kmeans_labels = kmeans.fit_predict(X)
```

```
# DBSCAN
```

```
dbscan = DBSCAN(  
  
    eps=0.2,
```

```
        min_samples=5

    )

    dbscan_labels = dbscan.fit_predict(X)

    # Side-by-side plots

    fig, axes = plt.subplots(

        1,

        2,

        figsize=(14, 6)

    )

    axes[0].scatter(

        X[:, 0],

        X[:, 1],

        c=kmeans_labels,

        cmap="viridis",

        s=25

    )
```

```
axes[0].set_title(  
  
    "K-Means Clustering"  
  
)
```

```
axes[0].set_xlabel("Feature 1")
```

```
axes[0].set_ylabel("Feature 2")
```

```
axes[1].scatter(  
  
    X[:, 0],  
  
    X[:, 1],  
  
    c=dbscan_labels,  
  
    cmap="viridis",  
  
    s=25  
  
)
```

```
axes[1].set_title(  
  
    "DBSCAN Clustering"  
  
)
```

```
axes[1].set_xlabel("Feature 1")
```

```
axes[1].set_ylabel("Feature 2")
```

```
plt.tight_layout()
```

```
plt.show()
```

Observation:

K-Means generally struggles with concentric circles, while DBSCAN can capture the density-based circular structure more naturally.

Question 44: Load the Iris Dataset and Plot the Silhouette Coefficient for Each Sample After KMeans Clustering.

Python Code:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_samples


# Load Iris

iris = load_iris()


X = iris.data


# Scale

X_scaled = StandardScaler().fit_transform(X)


# K-Means

model = KMeans(

    n_clusters=3,

    random_state=42,

    n_init=10

)
```

```
labels = model.fit_predict(

    X_scaled

)


# Silhouette values

sample_silhouette_values = silhouette_samples(

    X_scaled,

    labels

)


# Plot

plt.figure(figsize=(9, 7))


y_lower = 10


for cluster in range(3):

    values = sample_silhouette_values[
```

```
        labels == cluster

    ]

    values.sort()

    size_cluster = len(values)

    y_upper = (

        y_lower +

        size_cluster

    )

    plt.fill_betweenx(

        np.arange(

            y_lower,

            y_upper

        ),

        0,

        values
```

```
)
```

```
plt.text(
```

```
    -0.05,
```

```
    y_lower + 0.5 * size_cluster,
```

```
    str(cluster)
```

```
)
```

```
y_lower = y_upper + 10
```

```
plt.axvline(
```

```
    x=np.mean(
```

```
        sample_silhouette_values
```

```
    ),
```

```
    linestyle="--"
```

```
)
```

```
plt.xlabel(
```

```
    "Silhouette Coefficient"
```

```
)

plt.ylabel(

    "Cluster"

)

plt.title(

    "Silhouette Plot for Iris K-Means"

)

plt.show()
```

Observation:

Samples with higher silhouette values are generally better matched to their assigned clusters.

Question 45: Generate Synthetic Data Using `make_blobs` and Apply Agglomerative Clustering with 'average' Linkage. Visualize Clusters.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs

from sklearn.cluster import AgglomerativeClustering

# Generate data

X, y = make_blobs(

    n_samples=500,

    centers=4,

    cluster_std=1.0,

    random_state=42

)

# Agglomerative clustering

model = AgglomerativeClustering(

    n_clusters=4,

    linkage="average"
```

```
)
```

```
labels = model.fit_predict(
```

```
    X
```

```
)
```

```
# Plot
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(
```

```
    X[:, 0],
```

```
    X[:, 1],
```

```
    c=labels,
```

```
    cmap="viridis",
```

```
    s=30
```

```
)
```

```
plt.title(
```

```
    "Agglomerative Clustering - Average Linkage"
```

```
)
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.show()
```

Question 46: Load the Wine Dataset, Apply KMeans, and Visualize Cluster Assignments in a Seaborn Pairplot Using the First 4 Features.

Python Code:

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_wine
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import KMeans

# Load Wine dataset

wine = load_wine()

# First 4 features

X = wine.data[:, :4]

feature_names = wine.feature_names[:4]

# Scale

X_scaled = StandardScaler().fit_transform(X)

# K-Means

model = KMeans(

    n_clusters=3,

    random_state=42,

    n_init=10

)
```

```
labels = model.fit_predict(

    X_scaled

)


# DataFrame

df = pd.DataFrame(

    X,

    columns=feature_names

)


df["Cluster"] = labels.astype(str)


# Pairplot

sns.pairplot(

    df,

    vars=feature_names,

    hue="Cluster"

)
```

```
plt.show()
```

Observation:

The pairplot shows pairwise relationships among the first four Wine features and colors observations according to their K-Means cluster.

Question 47: Generate Noisy Blobs Using make_blobs and Use DBSCAN to Identify Both Clusters and Noise Points. Print the Count.

Python Code:

```
import numpy as np

from sklearn.datasets import make_blobs

from sklearn.cluster import DBSCAN

# Generate blobs

X, y = make_blobs(
```

```
        n_samples=500,

        centers=3,

        cluster_std=1.2,

        random_state=42

    )


# DBSCAN

model = DBSCAN(

    eps=0.7,

    min_samples=5

)


labels = model.fit_predict(

    X

)


# Cluster count excluding noise

clusters = set(labels)
```

```
if -1 in clusters:

    clusters.remove(-1)

cluster_count = len(clusters)

noise_count = np.sum(

    labels == -1

)

print(

    "Number of clusters:",

    cluster_count

)

print(

    "Number of noise points:",

    noise_count

)
```

```
print("\nCluster counts:")

unique, counts = np.unique(

    labels,

    return_counts=True

)

for label, count in zip(

    unique,

    counts

):

    if label == -1:

        print(

            f"Noise: {count}"

        )

    else:

        print(

            f"Cluster {label}: {count}"

        )
```

Observation:

DBSCAN labels noise as -1. The exact number of clusters and noise points depends on `eps`, `min_samples`, and the generated dataset.

Question 48: Load the Digits Dataset, Reduce Dimensions Using t-SNE, Then Apply Agglomerative Clustering and Plot the Clusters.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits

from sklearn.preprocessing import StandardScaler

from sklearn.manifold import TSNE

from sklearn.cluster import AgglomerativeClustering

# Load Digits

digits = load_digits()
```

```
X = digits.data

# Scale

X_scaled = StandardScaler().fit_transform(X)

# t-SNE

tsne = TSNE(

    n_components=2,

    random_state=42,

    perplexity=30

)

X_tsne = tsne.fit_transform(

    X_scaled

)

# Agglomerative Clustering

model = AgglomerativeClustering(
```

```
        n_clusters=10,

        linkage="ward"

    )

    labels = model.fit_predict(

        X_tsne

    )

    # Plot

    plt.figure(figsize=(10, 7))

    plt.scatter(

        X_tsne[:, 0],

        X_tsne[:, 1],

        c=labels,

        cmap="tab10",

        s=15

    )
```

```
plt.title(  
    "Agglomerative Clustering on Digits after t-SNE"  
)  
  
plt.xlabel("t-SNE 1")  
  
plt.ylabel("t-SNE 2")  
  
plt.show()
```

Observation:

The t-SNE representation makes the high-dimensional Digits dataset easier to visualize, and Agglomerative Clustering then groups the observations in this 2D representation.

QUICK REVISION TABLE

Algorithm / Concept	Main Idea
Unsupervised Learning	Finds patterns without target labels
K-Means	Centroid-based clustering
K-Means++	Better centroid initialization
Hierarchical Clustering	Builds hierarchy of clusters

Agglomerative Clustering	Bottom-up hierarchical clustering
Dendrogram	Shows cluster merging hierarchy
DBSCAN	Density-based clustering
<code>eps</code>	DBSCAN neighborhood radius
<code>min_samples</code>	Minimum neighborhood size for core points
Noise	DBSCAN label <code>-1</code>
Inertia	Within-cluster squared distance
Elbow Method	Helps choose K
Silhouette Score	Measures cohesion and separation
Single Linkage	Minimum inter-cluster distance
Complete Linkage	Maximum inter-cluster distance
Average Linkage	Average inter-cluster distance
Ward Linkage	Minimizes increase in within-cluster variance
PCA	Dimensionality reduction
t-SNE	Nonlinear dimensionality reduction mainly for visualization
StandardScaler	Standardizes features
MinMaxScaler	Scales features to a specified range

IMPORTANT FORMULAS

K-Means Inertia

Inertia=

$$\sum_{i=1}^n \|x_i - \mu_c\|^2$$

Silhouette Coefficient

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Where:

- $a(i)$ = average distance within the same cluster.
- $b(i)$ = lowest average distance to another cluster.

Silhouette Score Range

$$-1 \leq \text{Silhouette Score} \leq 1$$

GOOGLE COLAB IMPORTANT NOTES

1. Install/Import Required Libraries

Most libraries used in this assignment are already available in Google Colab.

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.cluster import DBSCAN
```

```
from sklearn.cluster import AgglomerativeClustering
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.manifold import TSNE
```

```
from sklearn.datasets import (
```

```
make_blobs,  
  
make_moons,  
  
make_circles,  
  
load_iris,  
  
load_wine,  
  
load_digits,  
  
load_breast_cancer  
)
```

2. K-Means

Use:

```
KMeans (  
  
    n_clusters=3,  
  
    random_state=42,  
  
    n_init=10  
)
```

3. DBSCAN

Use:

```
DBSCAN(  
  
    eps=0.5,  
  
    min_samples=5  
  
)
```

The best `eps` is dataset-dependent.

4. Agglomerative Clustering

Use:

```
AgglomerativeClustering(  
  
    n_clusters=3,  
  
    linkage="ward"  
  
)
```

For `ward`, Euclidean distance is required.

CONCLUSION

Clustering is an important **unsupervised learning** technique used to discover hidden structures and groups in datasets.

K-Means groups observations around centroids and works well when clusters are reasonably compact and spherical.

Hierarchical/Agglomerative Clustering creates a hierarchy of clusters and can be visualized using a **dendrogram**.

DBSCAN is a density-based method that can discover irregularly shaped clusters and identify noise points.

Silhouette Score and **Inertia/Elbow Method** help evaluate and select clustering solutions.

In real-world applications, clustering can be used for:

- Customer segmentation
- Market segmentation
- Image analysis
- Anomaly detection
- Document grouping
- Patient segmentation
- Recommendation systems
- Exploratory data analysis

Therefore, selecting the correct clustering algorithm depends on the **shape, scale, density, size, and nature of the dataset**.
